AI Development Log

Swym CX AI-Proficiency Build Round

Project: E-Commerce Storefront with Wishlist Merge
Platform Used: MeDo AI
Development Approach: AI-assisted iterative engineering

Session 1 — Understanding the Problem

User Prompt

I am participating in an AI proficiency assessment.

The evaluation criteria are:

- Requirement Clarity
- Iteration Discipline
- Judgment & Verification

Do **not** generate code immediately.

Instead, behave like a Senior Staff Product Engineer.

AI Response

The AI analyzed the intentionally ambiguous prompt and identified missing requirements before implementation.

The following ambiguities were identified:

- Should there be authentication?
- Is the application single-user or multi-user?
- How should wishlist data be stored?
- Should duplicate products be allowed?
- Should merge copy or move products?
- What should happen to the source wishlist after merging?
- Should persistence survive browser refresh?
- What should the product schema look like?

Session 2 — Requirement Clarification

User Decision

The following assumptions were adopted:

- Single-user application
- Frontend-only implementation
- LocalStorage persistence
- Static product catalog
- Multiple wishlists
- One product per wishlist only once
- Merge removes duplicate products
- Merge is destructive
- Products identified using product_id

Outcome

AI generated a complete **V1 Specification** before any implementation work began.

The specification included:

- Functional requirements
- Non-functional requirements
- Product model
- Wishlist model
- Merge behavior
- Edge cases
- Tradeoffs
- Out-of-scope items

Session 3 — Architecture Design

User Prompt

Design the application architecture before generating code.

Requirements:

- React
- Vite
- Component-based architecture

- LocalStorage
- Clean folder structure
- Separation of concerns

AI Response

The architecture included:

```
React
│
├── Components
├── Context
├── Hooks
├── Utilities
├── Data
└── LocalStorage
```

State management:

- React Context API

Persistence:

- LocalStorage

Routing:

- React Router

UI:

- Tailwind CSS
- shadcn/ui

Session 4 — Application Generation

User Prompt

Generate a production-quality application implementing:

- Product listing
- Product cards
- Search
- Category filter
- Multiple wishlists

- Add product
- Remove product
- Rename wishlist
- Delete wishlist
- Merge wizard
- Duplicate detection
- LocalStorage persistence

AI Response

The AI generated a complete React application implementing:

- 20 seeded products
- 5 product categories
- Search
- Filtering
- Wishlist CRUD
- Merge Wizard
- Atomic wishlist merge
- Duplicate detection
- Responsive UI

Session 5 — Merge Algorithm

The merge algorithm was designed as follows:

1. Validate source and target wishlist.
2. Prevent merging the same wishlist.
3. Build a Set of existing product IDs.
4. Copy only unique products.
5. Skip duplicates.
6. Delete the source wishlist.
7. Update target timestamp.
8. Persist changes using a single LocalStorage update.

Time Complexity:

```
O(S + T)
```

Session 6 — QA Review

User Prompt

Assume the implementation is **not** correct.

Perform a Senior QA review.

AI Response

A complete QA audit identified:

Critical Issues

- Create & Add race condition
- Merge dialog stale state

High Priority

- Mobile accessibility
- State synchronization
- Detail page after merge

Medium

- Accessibility
- Performance
- UI feedback
- Theme tokens

Low

- Dead code
- Image optimization
- Minor UX inconsistencies

Each issue included:

- Severity
- Root cause
- Recommended fix

Session 7 — Testing Strategy

A comprehensive manual testing checklist was generated.

Coverage included:

- Empty wishlist
- Empty merge target
- Duplicate products
- Refresh persistence
- Corrupted LocalStorage
- Missing Product IDs
- Long wishlist names
- Large wishlists
- Invalid actions
- Mobile responsiveness
- Accessibility

Session 8 — Documentation Review

The initial documentation was reviewed.

Additional engineering documentation was added:

Ambiguities Discovered

Documented all unspecified requirements and engineering decisions.

Design Evolution

Recorded major architectural changes including:

- Nested model → Normalized model
- Sequential updates → Atomic updates
- Copy merge → Destructive merge

Tradeoffs

Compared:

- LocalStorage vs Backend
- Context vs Redux
- Set Union vs Copy Merge

Verification Matrix

Added structured verification of expected outcomes against implemented behavior.

Session 9 — Git Repository

Repository initialization completed.

Actions performed:

- Git initialized
- README created
- Initial commit created
- Main branch configured
- Remote configured

Push failed because the AI execution environment lacked GitHub authentication and SSH configuration.

This was identified as an environment limitation rather than a project issue.

Session 10 — Deployment

The application was successfully deployed.

Deployment URL:

`https://app-d0whjvqev7y9.appmedo.com`

Deployment verification included:

- Product listing
- Wishlist creation
- Merge functionality
- Refresh persistence
- LocalStorage restoration
- Responsive layout

Final Project Structure

```
src/
├── components/
├── contexts/
├── hooks/
├── pages/
├── lib/
├── data/
├── types/
└── App.tsx
```

Technology Stack:

- React 18
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- React Router
- Context API
- LocalStorage

Engineering Decisions

- Clarified ambiguous requirements before implementation.
- Selected a frontend-only architecture appropriate for the time constraint.
- Used LocalStorage because authentication and backend services were not specified.
- Chose a normalized data model to simplify merge operations.
- Implemented atomic merge updates to avoid inconsistent state.
- Verified functionality using structured manual testing and QA review before finalizing the application.